



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

HTML (II parte)

Fabio Vitali

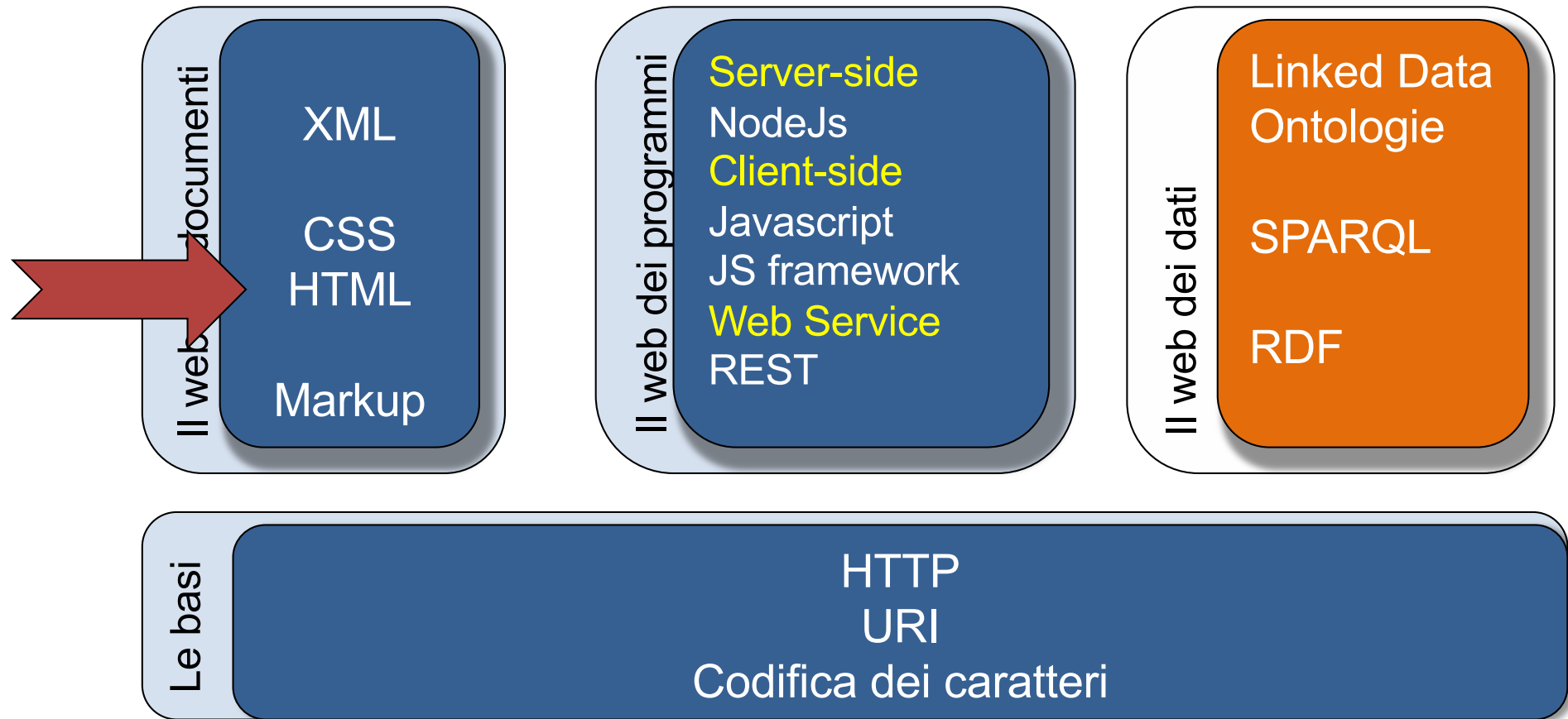
CdS in Informatica

Introduzione

Oggi esaminiamo in breve:

- HTML (concetti avanzati)

Argomenti delle lezioni





ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

HTML – altri aspetti

HTML – altri aspetti

Adesso parliamo di:

- *La struttura di un documento HTML*
- *Elementi inline*
- *Elementi di blocco e di lista*
- *Elementi generici*
- *Elementi di struttura*
- Link ed immagini
- Tipi di dati
- Entità predefinite
- Peculiarità sintattiche
- Tabelle
- Form
- Il contenuto dell'elemento HEAD



Link ipertestuali (anchors) (1)

I link sono definiti con elementi `<a>` (àncore nel documento).

`<a>` è sintatticamente un elemento inline (dentro ai blocchi, come `<b>`, `<i>`, ecc.). L'unica limitazione è che gli elementi `<a>` non possono annidarsi.

Attributi:

- **`href`**: specifica l'URI della destinazione. Quindi `<a href="xxx"> ... </a>` è l'ancora di partenza di un link.
- **`name`**: specifica un nome che può essere usato come ancora di destinazione di un link. Quindi `<a name= "xxx"> ... </a>` è l'ancora finale di un link.



Link ipertestuali (anchors) (2)

A link to the [whole document](#) called first

The following is a link to [a location](#) of the same document.

first.html →

```
<body>
  <p>A link to the <a href="second.html">
    whole document</a> called first</p>
  <p>The following is a link to <a
    href="second.html#location1"> a
    location</a> of the same document.</p>
</body>
</html>
```



Link ipertestuali (anchors) (2)

A link to the [whole document](#) called first

The following is a link to [a location](#) of the same document.

Beginning of the second document

... Much more content ...

A paragraph that contains a destination of a hypertext link.

... More content still

```
...html">  
st</p>  
a  
> a
```

second.html →

```
</body>  
<p>Beginning of the second document</p>  
... Much more content ...  
<p>A paragraph that contains <a  
    name="location1">a destination</a>  
    of a hypertext link.</p>  
... More content still  
</body>  
</html>
```


Immagini

Le immagini inline sono definite con l'elemento `<img>`. Formati tipici: jpeg, gif, png.

`<img>` è un elemento vuoto, completamente definito dai suoi attributi.

```

```

Attributi:

- **src** (required): l'URL della risorsa contenente l'immagine.
- **alt**: testo alternativo se l'immagine non è mostrata
- **width**: forza una larghezza in pixel per l'immagine.
- **height**: forza una altezza in pixel per l'immagine.



Immagini (2)

```
<body class="container-fluid">
  <h1>Images</h1>
  <p>Images are managed by the <img> element, which is empty (no content, no end tag).
  The <src> attribute is used to specify the URI of the image file. The <alt>
  attribute specifies an alternative text. The <height> attribute specifies the
  height in pixels.</p>
  <p>The image can be: </p>
  <ol>
    <li>... in the same folder as the HTML document:
      
    </li>
    <li>... in a different folder on the same server:
      
    </li>
    <li>... on a different server:
      
    </li>
  </ol>
</body>
```



Immagini (2)

Images

Images are managed by the `<img>` element, which is empty (no content, no end tag). The `src` attribute is used to specify the URI of the image file. The `alt` attribute specifies an alternative text. The `height` attribute specifies the height in pixels.

The image can be:

```
<body class="container-fluid">
  <h1>Images</h1>
  <p>Images are managed by the &lt;img>
  The <tt>src</tt> attribute is us
  attribute specifies an alternati
  height in pixels.</p>
  <p>The image can be: </p>
  <ol>
    <li>... in the same folder a
      
    <li>... in a different folde
      
  </ol>
</body>
```

1. ... in the same folder as the HTML document:



2. ... in a different folder on the same server:



3. ... on a different server:



Immagini (3)

```
<body class="container-fluid">
  <h1>Images - stretching</h1>
  <p>Specifying height and/or width lets control dimensions simply:
  <ol>
    <li>Specify one dimension: the image will be resized proportionally around that dimension.
      
    </li>
    <li>Specify both dimensions: the image will be resized regardless of the original
      proportions and will be stretched.
      
    </li>
    <li>Specify no dimensions: the image will be shown in the original dimensions.
      
    </li>
  </ol>
</body>
```



Immagini (3)

```
<body class="container-fluid">
```

```
<h1>Images - stretching</h1>
```

```
<p>Specifying height and/or width lets control dimensions simply:
```

```
<ol>
```

```
<li>Specify one dimension: the
```

```

```

```
<li>Specify both dimensions: t  
proportions and will be st
```

```

```

```
<li>Specify no dimensions: the
```

```

```

```
</ol>
```

```
</body>
```

Images - stretching

Specifying height and/or width lets control dimensions simply:

1. Specify one dimension: the image will be resized proportionally around that dimension.



2. Specify both dimensions: the image will be resized regardless of the original proportions and will be stretched.



3. Specify no dimensions: the image will be shown in the original dimensions.



Immagini responsive

Attraverso l'attributo `srcset` è possibile indicare più risorse per la stessa immagine, da usare alternativamente.

A seconda dello user agent scegliamo immagini con dimensioni (e quindi risoluzioni) diverse, e non dobbiamo affidarci al ridimensionamento algoritmico dell'immagine con l'inevitabile perdita di dettaglio.

```

```

- L'attributo `srcset` indica quali risorse usare e la dimensione "reale" di ciascuna risorsa.
- L'attributo `sizes` indica quali tipi di schermo associare a ciascuna risorsa.
- L'attributo `src` è un fallback nel caso che il browser non conosca `srcset`.



L'elemento <figure>

Le immagini sono dunque elementi vuoti inline.

In molte situazioni ho bisogno di un elemento complesso, di tipo blocco, magari con didascalia e marginatura controllabile.

Invece di usare un generico <div> con un generico è stato introdotto un elemento apposito, <figure>

Figure



A cake

```
<h1>Figure</h1>
<figure>
  
  <figcaption>A cake</figcaption>
</figure>
```

Embedding

Simili a `<img>`, permettono di arricchire la pagina HTML con molteplici tipi di contenuti alternativi.

- `<video>` per fare embedding di un video
- `<audio>` per fare embedding di un audio
- `<object>` per fare embedding di oggetti multimediali.
- `<embed>` elemento generico per fare embedding di oggetti multimediali. Un po' in disuso.
- `<iframe>` per fare embedding di una pagina HTML all'interno di un'altra pagina HTML.

```
<iframe title="Mappa OpenStreetMap – Bologna centro"
width="300" height="200" loading="lazy"
referrerpolicy="no-referrer"
sandbox="allow-scripts allow-same-origin"
src="https://www.openstreetmap.org/export/embed.html?
bbox=11.320%2C44.490%2C11.370%2C44.500">
</iframe>
```

Tabelle

Le tabelle vengono specificate riga per riga.

Di ogni riga si possono precisare gli elementi, che sono o intestazioni o celle normali.

Una tabella può anche avere una didascalia, un'intestazione ed una sezione conclusiva.

E' possibile descrivere insieme le caratteristiche visive delle colonne.



Tabelle (2)

```
<body>
  <h1>Tables</h1>
  <table border="1">
    <tr>
      <th>Salesman</th><th>Jan</th>
      <th>Feb</th> <th>Mar</th>
    </tr>
    <tr>
      <th>John Smith</th> <td>12000</td>
      <td>13000</td><td>15000</td>
    </tr>
    <tr>
      <th>Alice Green</th><td>7000</td>
      <td>9000</td><td>11000</td>
    </tr>
    <tr>
      <th>Hugh Brown</th><td>25000</td>
      <td>23000</td><td>30000</td>
    </tr>
  </table>
</body>
```

Tables

Salesman	Jan	Feb	Mar
John Smith	12000	13000	15000
Alice Green	7000	9000	11000
Hugh Brown	25000	23000	30000



Tabelle (3)

```
<body>
  <h1>Tables</h1>
  <table border="2" width="80%" cellpadding="5" cellspacing="0">
    <caption>Sales in first quarter</caption>
    <col style="background-color:#FFFFBB" width="70%">
    <col span="3" width="10%">
      <thead>
        <tr>
          <th>Salesman</th><th>Jan</th>
          <th>Feb</th> <th>Mar</th>
        </tr>
      </thead>
      <tfoot>
        <tr>
          <th>Totals</th><th>34000</th>
          <th>45000</th> <th>56000</th>
        </tr>
      </tfoot>
      <tbody>
        <tr>
          <th>John Smith</th> <td>12000</td>
          <td>13000</td><td>15000</td>
        </tr>
        <tr>
          <th>Alice Green</th><td>7000</td>
          <td>9000</td><td>11000</td>
        </tr>
        <tr>
          <th>Hugh Brown</th><td>25000</td>
          <td>23000</td><td>30000</td>
        </tr>
      </tbody>
    </table>
  </body>
```



Tabelle (3)

```
<body>
  <h1>Tables</h1>
  <table border="2" width="80%" cellpadding="5" cellspacing="0">
    <caption>Sales in first quarter</caption>
    <col style="background-color:#FFFFBB" width="70%">
    <col span="3" width="10%">
      <thead>
        <tr>
          <th>Salesman</th><th>Jan</th>
          <th>Feb</th> <th>Mar</th>
        </tr>
      </thead>
      <tfoot>
        <tr>
          <th>Totals</th><th>34000</th>
          <th>45000</th> <th>56000</th>
        </tr>
      </tfoot>
      <tbody>
        <tr>
          <th>John Smith</th> <td>12000</td>
          <td>13000</td><td>15000</td>
        </tr>
        <tr>
          <th>Alice Green</th><td>7000</td>
          <td>9000</td><td>11000</td>
        </tr>
        <tr>
          <th>Hugh Brown</th><td>25000</td>
          <td>23000</td><td>30000</td>
        </tr>
      </tbody>
    </table>
  </body>
```

Tables

Sales in first quarter

Salesman	Jan	Feb	Mar
John Smith	12000	13000	15000
Alice Green	7000	9000	11000
Hugh Brown	25000	23000	30000
Totals	34000	45000	56000

Un altro tipo di tabella

- Le celle possono occupare più righe o più colonne.
- Ogni `colspan` e `rowspan` toglie la cella corrispondente nella colonna o nella riga successiva.
- Utili per fare tabelle di layout (fortemente criticate per ragioni di accessibilità)



Un altro tipo di tabella (2)

```
<body>
  <h1>Tables</h1>
  <table border=2 cellpadding="5" cellspacing="0" width="70%">
    <tr>
      <td>First row <br> First column</td>
      <td colspan="2">First row<br>Second column</td>
    </tr>
    <tr>
      <td colspan="2">Second row <br> First column</td>
      <td rowspan="2">Second row <br> Third column</td>
    </tr>
    <tr>
      <td>Third row <br> First column</td>
      <td>Third row <br> Second column</td>
    </tr>
  </table>
</body>
```

Tables

First row First column	First row Second column	
Second row First column		Second row Third column
Third row First column	Third row Second column	

Form

Con i FORM si utilizzano le pagine HTML per inserire valori che vengono poi elaborati sul server. I FORM sono legati ad applicazioni server-side

Il browser raccoglie dati dall'utente con un form. Crea una connessione HTTP con il server, specificando una ACTION (cioè un applicazione che funga da destinatario) a cui fare arrivare i dati. Il destinatario riceve i dati, li elabora e genera un documento di risposta, che viene spedito, tramite il server HTTP, al browser.

I controlli tipati e nominati vengono usati per l'inserimento dei dati nei form: campi di inserimento dati, pulsanti, bottoni radio, checkbox, liste a scomparsa, ecc.



Form (2)

```
<h1>Form</h1>
<form method="get" action="http://www.site.com/serverside.py">
  <p>
    <label><i>Name:</i> <input type="text" name="name" value="John" size="15"></label>
    <label><i>Surname:</i> <input type="text" name="surname" value="Smith" size="25"></label>
  </p>
  <p>Gender:
    <label><input type="radio" name="gender" value="m" checked>Male</label>
    <label><input type="radio" name="gender" value="f">Female</label>
    <label><input type="radio" name="gender" value="x">Won't say</label>
  </p>
  <p>Likes:
    <label><input type="checkbox" name="likes" value="art" checked>Art</label>
    <label><input type="checkbox" name="likes" value="cin">Cinema</label>
    <label><input type="checkbox" name="likes" value="com">Comics</label>
    <label><input type="checkbox" name="likes" value="lit" checked>Literature</label>
    <label><input type="checkbox" name="likes" value="cui">Cuisine</label>
  </p>
  <p>
    <label>Nationality: <select name="nationality">
      <option value="">-- select one --</option>
      <option value="italian" selected>Italian</option>
      <option value="sanmarinese">San Marinese</option>
      <option value="taiwanese">Taiwanese</option>
      <option value="turkish">Turkish</option>
    </select></label>
  </p>
  <p><input type="submit" name="submit" value="ok">
    <input type="reset" name="cancel" value="cancel">
  </p>
</form>
```

Form (2)

```
<h1>Form</h1>
<form method="get" action="http://www.site.com/serverside.py">
  <p>
    <label><i>Name:</i> <input type="text" name="name" value="John" size="15"></label>
    <label><i>Surname:</i> <input type="text" name="surname" value="Smith" size="25"></label>
  </p>
  <p>Gender:
    <label><input type="radio" name="gender" value="m" checked>Male</label>
    <label><input type="radio" name="gender" value="f">Female</label>
    <label><input type="radio" name="gender" value="x">Won't say</label>
  </p>
  <p>Likes:
    <label><input type="checkbox" name="likes" value="art" checked>Art</label>
    <label><input type="checkbox" name="likes" value="cinema">Cinema</label>
    <label><input type="checkbox" name="likes" value="comics">Comics</label>
    <label><input type="checkbox" name="likes" value="literature">Literature</label>
    <label><input type="checkbox" name="likes" value="cuisine">Cuisine</label>
  </p>
  <p>
    <label>Nationality: <select name="nationality">
      <option value="">-- select one --
      <option value="italian">Italian
      <option value="sanmarino">San Marinese
      <option value="taiwanese">Taiwanese
      <option value="turkish">Turkish
    </select></label>
  </p>
  <p><input type="submit" name="submit" value="ok">
    <input type="reset" name="cancel" value="cancel">
  </p>
</form>
```

Form

Name: John

Surname: Smith

Gender: ☒ Male ☐ Female ☐ Won't say

Likes: ☒ Art ☐ Cinema ☐ Comics ☒ Literature ☐ Cuisine

Nationality: Italian

-- select one --

Italian

San Marinese

Taiwanese

Turkish

Form (3)

Gli elementi di un form sono:

- `<form>`: il contenitore dei widget del form
 - **Attributi:**
 - `method`: il metodo HTTP da usare (GET, POST)
 - `action`: l'URI dell'applicazione server-side da chiamare
- `<input>`, `<select>`, `<textarea>`: i widget del form.
 - **Attributi:**
 - `name`: il nome del widget usato dall'applicazione server-side per determinare l'identità del dato
 - `type`: il tipo di widget (input, checkbox, radio, submit, reset, etc.)
 - **N.B.:** Tutte le checkboxes e tutti i radio buttons dello stesso gruppo condividono lo stesso nome.
- `<button>`: un bottone cliccabile (diverso dal submit)
- `<label>`: la parte visibile del widget.



Interactive content: form

HTML LS introduce molte novità per velocizzare, semplificare e controllare l'inserimento dei dati da parte dell'utente

L'oggetto **input** è arricchito con molti attributi che permettono di controllare il focus e aggiungere suggerimenti agli utenti:

- **placeholder**: contiene una stringa che sarà visualizzata in un campo di testo se il focus non è su quel campo
- **required**: il form non può essere sottomesso senza una valore per questo elemento.
- **readonly**: è un elemento del form non modificabile.
- **list**: il valore deve appartenere ad uno dei valori ammessi (specificati in un elemento **<datalist>** associato.

```
<input type="text" list="province">  
<datalist id="province">  
  <option value="BO">Bologna</option>  
  <option value="MO">Modena</option>  
  <option value="FC">Forlì Cesena</option>  
</datalist>
```



Alcuni ulteriori tipi di input

In HTML 5 sono introdotti molti nuovi tipi per l'oggetto **input**. I browser forniscono widget diversi nell'interfaccia in base al tipo del campo.

- **email**: in fase di validazione il browser verifica se il testo inserito contiene il simbolo '@' e il dominio è (sintatticamente) corretto
- **url**: si verifica che il testo inserito segue le specifiche degli URL
- **number**: il browser visualizza bottoni per incrementare/decrementare il valore. E' possibile specificare il valore minimo, massimo e l'unità di modifica in altri attributi di **input**.
- **range**: il browser visualizza uno slider per incrementare o decrementare un valore numerico. E' possibile specificare il valore minimo, massimo e l'unità di modifica.
- **date**: richiede al browser la visualizzazione di un calendario tra cui selezionare una data. Esistono vari tipi collegati come **month**, **week**, **time** (orario), etc.
- **search**: testo renderizzato in modo diverso su alcuni browser (iPhone)
- **color**: usato per mostrare una tavolozza di colori, da cui selezionare un codice *RGB*



Esempio

<p>Nome: <input name="nome" type="text" placeholder="Inserisci qui il tuo nome" size="40" autofocus></p>

<p>Cognome: <input name="cognome" size="40" placeholder="Inserisci qui il tuo cognome"></p>

<p>Email:<input name="mail" type="email"></p>

<p>Lezioni: <input type="number" min="5" max="25" step="1" value="1"></p>

<p>Livello: <input type="range" name="level" min="1" max="10" step="1" value="3"></p>

<p>Data Inizio: <input name="birth" type="date" value="2011-04-01"></p>

<p><input type="submit" value="Iscriviti"></p>



Esempio

<p>Nome: <input name="nome" type="text"
placeholder="Inserisci qui il tuo nome" size="40"
autofocus></p>

<p>Cognome: <input name="cognome" size="40"
placeholder="Inserisci qui il tuo cognome"></p>

<p>Email: <input name="mail" type="text"></p>

<p>Lezioni: <input type="number"
step="1" value="1"></p>

<p>Livello: <input type="range"
max="10" step="1" value="3"></p>

<p>Data Inizio: <input name="bi"
value="2011-04-01"></p>

<p><input type="submit" value="Iscriviti"></p>

The screenshot shows a web form with the following elements:

- Nome:** A text input field with the placeholder "Inserisci qui il tuo nome".
- Cognome:** A text input field with the placeholder "Inserisci qui il tuo cognome".
- Email:** A text input field.
- Lezioni:** A number input field with a value of 1 and up/down arrows.
- Livello:** A range slider with a blue knob positioned at 3.
- Data Inizio:** A date input field showing "2011-04-01" with a calendar icon.
- Iscriviti:** A submit button.
- Calendar:** A calendar for April 2011. The days of the week are listed in the header. The dates 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, and 31 are displayed. The date 1 is highlighted in grey.



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Approfondimenti sulla sintassi di HTML

Attributi globali

Sono attributi globali quelli definiti su tutti gli elementi del linguaggio HTML.

Essi costituiscono attributi di qualificazione e associazione globale degli elementi. Per lo più per CSS e link ipertestuali.

- **Id**: un identificativo unico (su tutto il documento)
- **Class**: una lista (separata da spazi) di nomi di classe (per attribuzione semantica e di stile CSS)
- **Style**: un breve stile CSS associato al singolo elemento
- **Title**: un testo secondario associato all'elemento (per accessibilità e informazioni aggiuntive)



Attributi globali (2)

```
<!DOCTYPE html>
<html>
  <head>
    <title>Table elements </title>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <style>
      #p1 { font-family: "Times New Roman", "Times", serif;}
      #p2, #p3 { font-family: "Verdana", "Arial", sans-serif; }
      .first { color: blue; }
      .second {color: green; }
      .special {font-weight: bold; }
      .notspecial {font-weight: normal; }
    </style>
  </head>
  <body>
    <p id="p1" class="first special">This is the first paragraph of a short text.</p>
    <p id="p2" class="second">This is the <span id="s1" class="first">second</span>
    paragraph of the same text.</p>
    <p id="p3" class="special">This is the <span id="s2" class="notspecial"
    title="terzo!">third</span> paragraph of the same text.</p>
  </body>
</html>
```

Attributi globali (2)

This is the first paragraph of a short text.

This is the second paragraph of the same text.

This is the third paragraph of the same text.

terzo!

```
<!DOCTYPE html>
<html>
  <head>
    <title>Table elements <
    <meta charset="utf-8">
    <meta name="viewport" c
    <style>
      #p1 { font-family: "Times New Roman", "Times", serif;}
      #p2, #p3 { font-family: "Verdana", "Arial", sans-serif; }
      .first { color: blue; }
      .second {color: green; }
      .special {font-weight: bold; }
      .notspecial {font-weight: normal; }
    </style>
  </head>
  <body>
    <p id="p1" class="first special">This is the first paragraph of a short text.</p>
    <p id="p2" class="second">This is the <span id="s1" class="first">second</span>
    paragraph of the same text.</p>
    <p id="p3" class="special">This is the <span id="s2" class="notspecial"
    title="terzo!">third</span> paragraph of the same text.</p>
  </body>
</html>
```

Attributi globali: altri tipi

Gli attributi i18n (*internationalization*) garantiscono l'internazionalizzazione del linguaggio e la coesistenza di alfabeti

- **lang**: il linguaggio umano (BCP 47 di IETF a due caratteri: it, en, fr, etc. o quattro caratteri (en-gb)
- **dir**: uno dei due valori ltr (*left-to-right*) o rtl (*right-to-left*) per indicare la direzione di flusso secondario del testo.

Gli attributi di interattività permettono di dare indicazioni su come rendere interattivi gli elementi corrispondenti (per form o accessibilità):

- **accesskey** (tasto di tastiera per focalizzare l'elemento)
- **autofocus** (l'elemento viene focalizzato all'avvio)
- **tabindex** (l'elemento è raggiungibile via tab dopo *n* pressioni)
- **inputmode** (quale tastiera visualizzare quando l'elemento è focalizzato)

Gli attributi di evento permettono di associare script a particolari azioni sul documento e sui suoi elementi:

- **Onclick, ondoubleclick, onmouseover, onkeypress, ecc.**



Attributi data-

Attributi personalizzati che possono essere utilizzati da applicazioni e script Javascript senza inquinare lo spazio dei nomi.

Hanno la forma data-xxx. Ad es. in Bootstrap:

```
<button id="first" data-bs-toggle="modal" data-bs-target="#exampleModal">  
  show modal  
</button>
```

Non tutti i caratteri sono leciti, le maiuscole non sono lecite, i trattini vengono automaticamente convertiti in maiuscole nel dataset.

- Accedibile via CSS esattamente come scritto:
 - `button[data-bs-toggle] { color: blue; }`
- Accedibile via Javascript con l'array dichiarativo `element.dataset.xxx` .
 - `let a = document.getElementById('first').dataset['bsToggle']`
`a` vale "modal"



Attributi ARIA

HTML è accessibile per default.

Tuttavia nell'uso comune esistono tante cattive abitudini che creano pagine web perfettamente funzionanti per i non disabili ma impossibili da usare per i disabili.

Ad esempio:

```
<button class="button" onclick="doSomething()">Clicca qui</button>
```

e

```
<span class="button" onclick="doSomething()">Clicca qui</span>
```

possono essere resi visivamente identici ma il secondo non è accessibile dai non-vedenti.

La specifica WAI-ARIA (*Web Accessibility Initiative - Accessible Rich Internet Applications*) fornisce molte tecniche per rendere accessibili pagine HTML

- Attributo **role** per caratterizzare la semantica attesa dell'elemento
- Attributo **tabindex** per specificare la possibilità di selezionare l'elemento con la tastiera
- Attributi **aria-*** per specifiche funzionalità ai vari elementi a seconda del loro scopo.

Dedicheremo due lezioni ad ARIA nella seconda metà del corso. Questo elemento è accessibile:

```
<span role="button" class="button" onclick="doSomething()"
tabindex="0">Clicca qui</span>
```



Entità in HTML

HTML definisce un certo numero di entità per quei caratteri che sono proibiti perché usati in HTML (<, >, &, “, ecc.) o proibiti perché non presenti nell'ASCII a 7 bit.

Ad esempio:

– amp	&	quot	“
– lt (less than)	<	gt (greater than)	>
– Aelig	Æ	Aacute	Á
– Agrave	À	Auml	Ä
– aelig	æ	aacute	á
– agrave	à	auml	ä
– ccedil	ç	ntilde	ñ
– reg	®	nbsp (non-breaking space)	

ecc.

HTML	Shown as
<code>libert&agrave;</code>	libertà
<code>&copy; 2018 Fabio Vitali</code>	© 2018 Fabio Vitali
<code>Fran&ccedil;ois</code>	François
<code>5 &gt; 3 &amp; 5 &lt; 12</code>	5 > 3 & 5 < 12

In aggiunta, si possono usare entità numeriche:

- `Libertà` Libertà
- `你好` 你好



Tipi di dati: i colori

In molte situazioni (sfondi, caratteri, ecc.) è possibile specificare un colore. HTML fornisce due modi per farlo:

- **Codice RGB prefissato da un carattere di hash.** Si usano due caratteri esadecimali ciascuno per esprimere la quantità di Rosso, Verde e Blu del colore (00 significa assenza, FF significa presenza massima). E' possibile descrivere 16 milioni di colori diversi.
- **Nome:** sono definiti 16 nomi di colori: black, silver, gray, white, maroon, red, purple, fuchsia, green, lime, olive, yellow, navy, blue, teal, aqua. N.B. Microsoft definisce 256 nomi di colori, che IE accetta, ma che Netscape ignora. Questo causa incompatibilità tra i browser.

Tuttavia HTML 4 deprecia l'uso esplicito di colori nel documento HTML, e suggerisce di usare i fogli di stile.

- `<font color="#FF0000">testo in rosso</font>`
- `<body bgcolor="#008080">sfondo teal</body>`
- `<td bgcolor="yellow">sfondo giallo</td>`



Tipi di dati: le lunghezze

HTML usa le lunghezze per tutti gli oggetti con una presenza sulla pagina di dimensioni determinate (immagini, tabelle, frame, ecc.) Si usano tre tipi di lunghezze:

- ***Pixel***: una dimensione in punti di schermo. È un numero assoluto.

```

```

- ***Percentuali***: una dimensione in proporzione alla dimensione del contenitore. È un numero seguito da un carattere %.

```
<table width="100%">...</table>
```

- ***Multi-lunghezze***: una sequenza di valori di lunghezza. In questo caso il carattere '*' divide la quantità restante in parti uguali (dopo aver tolto le lunghezze esplicite in pixel e percentuale). È una lista separata da virgole di valori numerici assoluti. La si usa ad esempio nei gruppi di colonne di una tabella.

```
<colgroup width="150,25%,*,*">
```



Peculiarità sintattiche di HTML

Maiuscolo/minuscolo:

- HTML non è sensibile al maiuscolo/minuscolo (XHTML lo è e vuole tutto in minuscolo). Editor di antica concezione scrivevano i tag in maiuscolo (per meglio distinguerli dal testo), i nuovi in minuscolo (per meglio transitare verso XHTML).

Whitespace

- HTML collassa tutti i caratteri di whitespace (SPACE, TAB, CR, LF) in un unico spazio. Questo permette di organizzare il sorgente in modalità leggibile senza influenzare la visualizzazione su browser.
- L'entità permette di inserire spazi non collapsabili.

Estensioni del linguaggio

- I browser permettono (ignorandoli), mentre i validatori segnalano elementi e attributi che non appartengono al linguaggio

`<p pluto="paperino"><pippo>testo</pippo></p>`

Bizzarrie sintattiche

- Dovute perlopiù al fatto che la sintassi HTML deriva da SGML, e SGML permette queste cose. Viceversa, XHTML è un documento XML, e queste cose non le permette.



Bizzarrie sintattiche di HTML

(rimosse in XHTML, rimesse in HTML LS) (1)

Virgolette negli attributi

- Tutti i valori degli attributi che sono TOKEN (cioè parole di caratteri e numeri senza spazi e che inizino con un carattere) possono essere posti senza virgolette

```
<p align=center>testo al centro</p>
```

dovrebbe essere

```
<p align="center">testo al centro</p>
```



Bizzarrie sintattiche di HTML

(rimosse in XHTML, rimesse in HTML LS) (2)

Assenza dei tag

Alcuni tag (ad esempio <HTML>, <BODY>), possono essere omessi

**<p>Questo è un documento valido in HTML,
ma non in XHTML</p>**

dovrebbe essere

<html>

<body>

**<p>Questo è un documento valido sia
in HTML, sia in XHTML</p>**

</body>

</html>



Bizzarrie sintattiche di HTML

(rimosse in XHTML, rimesse in HTML LS) (3)

Omissibilità del tag di chiusura

Di alcuni elementi (ad esempio, <P>, , <OPTION>) può essere omesso il tag finale (è implicitamente dedotto dal contesto)

```
<ul>  
  <li>Primo elemento  
  <li>Secondo elemento  
  <li>Terzo elemento  
</ul>
```

dovrebbe essere

```
<ul>  
  <li>Primo elemento</li>  
  <li>Secondo elemento</li>  
  <li>Terzo elemento</li>  
</ul>
```



Bizzarrie sintattiche di HTML

(rimosse in XHTML, rimesse in HTML LS) (4)

Attributi di solo valore

Alcuni attributi (nowrap, selected, etc.) vengono omessi, scrivendo solo il valore equivalente:

```
<select>  
  <option>prima voce  
  <option selected>seconda voce  
  <option>terza voce  
</select>
```

dovrebbe essere

```
<select>  
  <option>prima voce</option>  
  <option selected="selected">seconda  
voce</option>  
  <option>terza voce</option>  
</select>
```





ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Altre caratteristiche

- `<head>`
- `<object>`
- `<canvas>`
- **web components**

I tag di `<head>`

L'elemento `<head>` contiene delle informazioni che sono rilevanti per tutto il documento. Esse sono:

- `<title>`: il titolo del documento
- `<link>`: link di documenti a tutto il documento
- `<script>`: librerie di script
- `<style>`: librerie di stili
- `<meta>`: meta-informazioni sul documento
- `<base>`: l'URL da usare come base per gli URL relativi



I tag di `<head>`:

`<title>`: il titolo del documento

L'elemento `<title>` contiene semplice testo (non può contenere elementi HTML) che definiscono il titolo del documento.

Il contenuto dell'elemento `<title>` ha tre scopi:

- Viene posto come titolo della finestra nei sistemi operativi a finestra
- Viene utilizzato come nome illustrativo della pagina nei bookmark
- Viene utilizzato come nome illustrativo della pagina nei motori di ricerca. Inoltre i motori di ricerca trattano con più importanza il contenuto dell'elemento `title` rispetto al resto del contenuto della pagina.



I tag di `<head>`:

`<script>`: codice e dati interni ed esterni

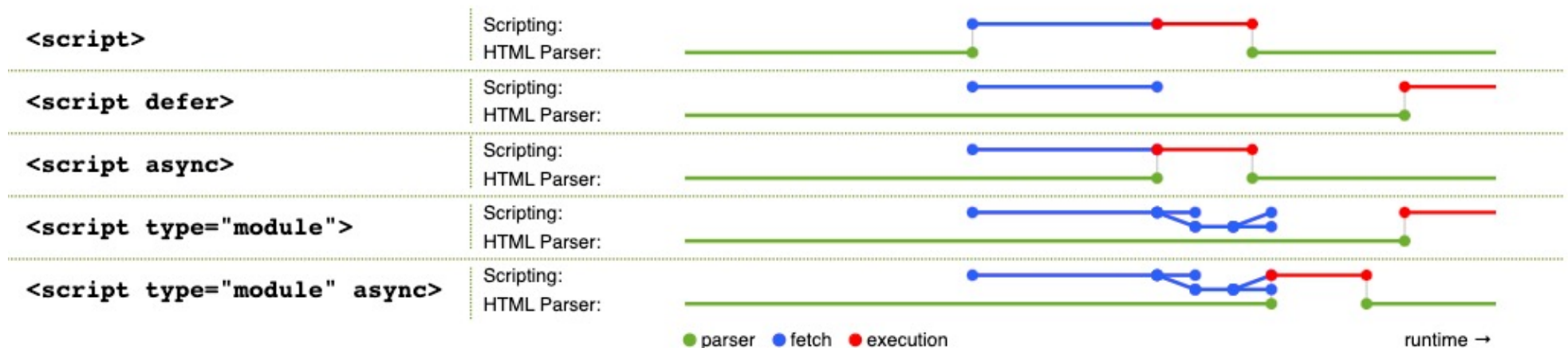
Con il tag `<script>` si definiscono blocchi di codice di un linguaggio di script. A volte può esser utile mettere questo codice in una risorsa a parte (es. condivisa), e riferirvi esplicitamente.

```
<script type="application/javascript"> ... </script>
```

```
<script type="application/javascript" src="lib/script.js"></script>
```

Attributi:

- `type="module"`: usa la sintassi JS dei moduli introdotta per facilitare la componentizzazione degli script Javascript (cambia la sintassi JS)
- `defer`: scarica lo script in parallelo ma lo esegue alla fine del caricamento
- `async`: scarica lo script in parallelo ma lo esegue appena arriva disponibile



I tag di **<head>**:

<link>, **<style>**: rappresentazione dello stile

Con i tag **<link>** e **<style>** si possono definire, rispettivamente, blocchi blocchi interni ed esterni di stili di un linguaggio di stylesheet.

A volte può esser utile mettere esternamente queste specifiche, e riferirvi esplicitamente.

In questo caso si usa il tag LINK, che permette di creare un link esplicito al documento esterno di script e/o di stili.

```
<link rel="stylesheet" type="text/css" href="style1.css">
```



I tag di <head>:

<meta>: meta-informazioni (1)

Le meta-informazioni sono informazioni sul documento, piuttosto che informazioni del documento. Il tag <meta> è un meccanismo generale per specificare meta-informazioni sul documento HTML.

Ci sono tre tipi di meta-informazioni definibili con il tag <meta>:

- La codifica caratteri utilizzata nel file:

<meta charset="utf-8">

- Intestazioni HTTP: la comunicazione HTTP fornisce informazioni sul documento trasmesso, ma il suo controllo richiede accesso al server HTTP. Con il tag META si può invece fornire informazione “stile-HTTP” senza modifiche al server. - **SCONSIGLIATA**

<meta http-equiv="expires" content="Sat, 23 Mar 2019 14:25:27 GMT">

- Altre meta-informazioni: i motori di ricerca usano le meta-informazioni (ad esempio “Keyword”) per organizzare al meglio i documenti indicizzati.



I tag di <head>:

<meta>: meta-informazioni (2)

```
<html>
<head>
  <meta charset="utf-8">
  <title>An example of a head element with meta tags</title>
  <meta name="DC.Creator" content="Fabio Vitali">
  <meta name="DC.Title" content="Using meta tags">
  <meta name="DC.Date" content="2019-03-04">
  <meta name="DC.Format" content="text/html">
  <meta name="DC.Language" content="en">
  <meta name="viewport" content="width=500, initial-scale=1">
</head>
<body>
  ...
</body>
</html>
```



I tag di `<head>`:

`<meta>`: meta-informazioni (3)

```
<meta name="viewport" content="width=500, initial-scale=1">
```

Il `<meta name="viewport">` è una proposta Apple in corso di standardizzazione per rendere pagine non progettate per il mobile leggibili anche su un device mobile.

Attraverso questo `<meta>` imponiamo al device in questione (tipicamente uno smartphone) le caratteristiche da utilizzare per il viewport invece di quelle fisiche, in modo che le pagine non risultino illeggibili sui browser e alla peggio scrollando si acceda alla parte di documento nascosto.

Nell'esempio sopra, si impone di considerare la larghezza della pagina a 500 px e considerare i pixel CSS della pagina tali e quali ai pixel dello schermo.



I tag di **<head>**:

<base>: URL relativi ed assoluti

Ogni documento HTML visualizzato in un browser ha associato un URL. Questo può appartenere allo schema di naming http://, ftp://, o anche file://. Tipicamente sono schemi gerarchici.

Spesso accade che esistano degli oggetti dipendenti dalla pagina (immagini, stili, script, applet, link a pagine secondarie, ecc.), che appartengono allo “stesso dominio” della pagina di partenza.

E' data allora possibilità, nello specificare l'URL della risorsa secondaria, di affidarsi ad un URL relativo, che si basa sull'URL del documento di partenza.





ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Altri dettagli su HTML

<object>: embedded content

HTML5 estende notevolmente le possibilità di integrazione di contenuti multimediali nelle pagine

Elementi come **<canvas>**, **<audio>**, **<video>**, **<math>** permettono di “includere” contenuti con i quali, ed è questa la vera novità e il punto di forza, è possibile interagire in modo avanzato

Il modello ad eventi di DOM è esteso con eventi specifici che permettono la costruzione di applicazioni sofisticate client-side. Agli eventi di aggiungono sofisticate API di manipolazione degli oggetti

Guardiamo in particolare:

- Canvas
- Embedding di contenuti audio e video



Canvas

Una delle più importanti innovazioni di HTML 5 è la possibilità di disegnare direttamente sulla pagina interagire con gli oggetti multimediali

L'elemento **<canvas>** definisce un'area rettangolare in cui disegnare direttamente immagini bidimensionali e modificarle in relazione a eventi, tramite funzioni Javascript.

La larghezza e l'altezza del canvas sono specificati tramite gli attributi **width** e **height** dell'elemento **<canvas>**.

Le coordinate **(0,0)** corrispondono all'angolo in alto a sinistra.



HTML Canvas 2D Context

Gli oggetti non sono disegnati direttamente sul canvas ma all'interno del **contesto**, recuperato tramite un metodo Javascript dell'elemento `<canvas>` chiamato `getContext()`

Questo metodo è parte di una vasta libreria utile per disegnare figure, colorarle, trasformarle, etc.

Questa API, inizialmente nota come *canvas API* e tuttora integrata con questo nome in “HTML Living Standard”, è diventata una specifica W3C distinta da HTML, anche se usa le stesse definizioni core, le stesse classi e lo stesso background di riferimento

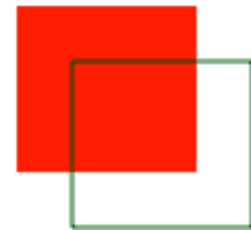
“HTML Canvas 2D Context” è una Recommendation W3C dal 19 Novembre 2015



Esempio

```
function draw(){  
    var canvas = document.getElementById('c1');  
    if (canvas.getContext){  
        var ctx = canvas.getContext('2d');  
        ctx.fillStyle = "rgb(255,0,0)";  
        ctx.fillRect (20, 20, 65, 60);  
        ctx.strokeStyle = "rgb(0, 100, 20)";  
        ctx.strokeRect (40, 40, 65, 60);  
    }  
}
```

```
<canvas id="c1" onLoad="draw();"   
    width="175" height="175">  
</canvas>
```



Video e audio

HTML 5 permette di includere video in una pagina senza richiedere plug-in esterni (Flash, Real Player, Quicktime)

L'elemento `<video>` specifica un meccanismo generico per il caricamento di *file* e *stream* video, più alcune proprietà DOM per controllarne l'esecuzione

Ogni elemento `<video>` in realtà può contenere diversi elementi `<source>` che specificano diversi file, tra i quali il browser sceglie quello da eseguire.

L'elemento `<audio>` è usato allo stesso modo per i contenuti sonori

Non esiste tuttavia una codifica universalmente accettata ma è necessario codificare il video (o audio) in più formati, per renderlo realmente *cross-browser*.



Esempio

```
<video width="400px" controls autoplay>  
  <source src="video.mp4" type="video/mp4"  
codecs="avc1.42E01E, mp4a.40.2">  
  <source src='video.ogv' type='video/ogg'  
codecs="theora, vorbis">  
  <track kind="subtitles" src="video.en.vtt"  
srclang="en" label="English">  
  <track kind="subtitles" src="video.it.vtt"  
srclang="it" label="Italian">  
</video>
```



WebComponents

Prendendo ispirazione dai sistemi a componenti come Angular, React e Vue, anche la sintassi standard di HTML ha introdotto i web component.

- Custom element: posso estendere il vocabolario degli elementi di HTML con quelli che più mi piacciono.
- Shadow DOM: posso creare un mini-DOM protetto a cui non si applicano script, stili e eventi del documento principale
- HTML template: posso associare ad un custom element un frammento HTML che va automaticamente a rimpiazzare il custom element di riferimento.

Rispetto ai sistemi a componenti tradizionali però è un modello un po' macchinoso. Ne riparlamo più avanti.



WebComponents

Per esempio per fare una dialog box dovrei scrivere qualcosa tipo:

```
<div class="modal" tabindex="-1">
  <div class="modal-header">
    <h5 class="modal-title">Hello world</h5>
    <button type="button" class="btn-close"
      aria-label="Close"></button>
  </div>
  <div class="modal-body">Some content</div>
  <div class="modal-footer">
    <button type="button">Close</button>
  </div>
</div>
```



WebComponents

Con i web component posso semplicemente scrivere:

```
<my-modal title="Hello world">Some content</my-modal>
```

il che può essere usato con un template come:

```
<div class="modal" tabindex="-1">  
  <div class="modal-header">  
    <h5 class="modal-title"><slot name="title"></h5>  
    <button type="button" class="btn-close"  
      aria-label="Close"></button>  
  </div>  
  <div class="modal-body"><slot></slot></div>  
  <div class="modal-footer">  
    <button type="button">Close</button>  
  </div>  
</div>
```



Conclusioni

In questa lezione abbiamo cercato di fare chiarezza tra:

- HTML
- (X)HTML 5
- HTML 4.x
- XHTML 1.0



Una piccola prova

<https://www.fabioitali.it/TW/2025/form/>

Usate per le vostre prove l'action

<https://www.fabioitali.it/TW/2025/form/doResponse.php>



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Riferimenti

The HTML Living Standard:

<http://www.whatwg.org/html>

W3C HTML5:

<http://www.w3.org/TR/html5/>

D. Raggett, A. Le Hors, I. Jacobs, *HTML 4.01 Specification*, W3C Recommendation 24 December 1999,

<http://www.w3.org/TR/html401>

M. Altheim et alii, *Modularization of XHTML*, W3C Recommendation 10 April 2001, W3C Recommendation 10 April 2001,

<http://www.w3.org/TR/xhtml1-modularization>

M. Altheim, S. McCarron, *XHTML™ 1.1 - Module-based XHTML*, W3C Recommendation 31 May 2001,

<http://www.w3.org/TR/xhtml11>

J. Axelsson et alii, *XHTML™ 2.0*, W3C Working Draft 31 January 2003,

<http://www.w3.org/TR/2003/WD-xhtml2-20030131>





ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Fabio Vitali

Corso di tecnologie web

Fabio.vitali@unibo.it

www.unibo.it